Joe: It looks to me like we have a few options there. We can either write our own password-strength algorithm or use an existing one. There are pros and cons to each approach, but using an existing library would probably be faster to implement.

You: Let's keep our options open: we don't know enough about what will suit us best to commit to a particular solution just yet, so let's do what we can to learn more. We'll see if we can find a good existing library and experiment with it, but we should integrate it in a way that we can easily switch to another library or our own custom solution later on if we aren't happy with the one we find.

Joe: Raj, I should be able to build a version of this using a couple of possible libraries by Thursday that you can play around with. Based on your feedback, we can fine-tune the solution we pick, or try out another one.

Susan: We can use this table of sample passwords as a starting point for the acceptance criteria. We may refine it or add new examples later on as we learn more about what we can do.

Password	Strength	Acceptable
secret	Weak	No
password	Weak	No
password1	Weak	No
aBcdEfg1	Weak	No
qwertY12	Weak	No
dJeZDip1	Medium	Yes
SeagullHedgehog	Strong	Yes
SeagullHedgehogCatapult	Very Strong	Yes

You've now gone from having what appeared to be a clear and simple set of requirements to discovering that the real requirements are not quite so obvious. What initially appeared to be business rules requested by the user turned out to be just one possible solution to the underlying business problem of ensuring that members have secure passwords. The team identified several possible approaches, but deferred choosing a specific option until they knew more about what solution they would use. And they identified a strategy that the team could use to get useful feedback from the business; this will help them select the most appropriate solution.

This sort of situation occurs often in software development; it's important to know what you don't know, and to cater for it in your decisions. In fact, many of the design decisions the team made in this example are founded on two important BDD concepts: Real Options and Deliberate Discovery.